1. How super () Handles Multiple Inheritance?

In Python, the super () function is used to call a method from a parent (or superclass) in the class hierarchy. When multiple inheritance is involved, super () doesn't just refer to one specific superclass, it follows Python's **Method Resolution Order (MRO)**.

MRO (Method Resolution Order)

MRO is the order in which Python looks for a method or attribute when it's called on an object in a class hierarchy especially when multiple inheritance is involved.

Python uses the **C3 Linearization algorithm** to compute MRO, this ensures:

- A consistent and predictable order
- That each class appears only once in the MRO list
- That child's classes come before parent classes

inheritance.py > ...

```python
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    def greet(self):
        print("Hello from B")
        super().greet()

class C(A):
    def greet(self):
        print("Hello from C")
        super().greet()

class D(B, C):  # D inherits from B and C
    def greet(self):
        print("Hello from D")
        super().greet()

d = D()
d.greet()

```

```
PS D:\ITI\Python> python inheritance.py
Hello from D
Hello from B
Hello from C
Hello from A
PS D:\ITI\Python>
```

How it works:

- super (). greet () in class D follows MRO: D → B → C → A
- So super () doesn't just jump to B's parent—it respects the MRO chain

So super () in multiple inheritance is not about the nearest superclass, but about the next class in MRO. This allows cooperative multiple inheritance, where all classes in the hierarchy can have their methods called properly

2. Method Overriding: Person and Employee Have the Same Method

Suppose you have a class hierarchy like this:

inheritance.py > ...

```python
class Person:
    def __init__(self,name):
        self.name=name
        print(f"person initailized with name {name}")
    def position(self):
        print("person")

class Employee(Person):
    def __init__(self,name,employee_id):
        super(Employee,self).__init__(name)
        self.employee_id=employee_id
        print(f"person initailized with name {name} and id {employee_id}")
    def position(self):
        print("employee")


class Teacher(Employee,Person):
    def __init__(self,name,employee_id,salary):
        self.salary=salary
        super(Teacher,self).__init__(name,employee_id)
        print(f"person initailized with name: {name} and id: {employee_id} and salary= {salary}")

teacher=Teacher("A","2d5",4500)
teacher.position()


```

The Output:

```
PS D:\ITI\Python> python inheritance.py
person initailized with name A
person initailized with name A and id 2d5
person initailized with name: A and id: 2d5 and salary= 4500
employee
PS D:\ITI\Python>
```

Python looks for the position method using the MRO for Teacher. You can view it using:

```
27    print(Teacher.__mro__)
28

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS D:\ITI\Python> python inheritance.py
employee
(<class '__main__.Teacher'>, <class '__main__.Employee'>, <class '__main__.Person'>, <class 'object'>)
PS D:\ITI\Python>
```

super () is MRO-aware and is used to ensure a predictable and consistent method-calling order in multiple inheritance.

When two parent classes define the same method, Python resolves it using the MRO, so the method in the first class listed in the inheritance will be called—unless super () is used cooperatively.